



UNIVERSITÀ DEGLI STUDI
DI GENOVA

Decentralized System

ASSIGNMENT 2 (NFT – Digital Asset: Frontend)

Name: Fazeel Ahmed

Matricola: S5640427

Group: Single Member

Submitted To:

Professor Matteo Dell'Amico

Professor Marina Ribaudo

Contents

Frontend Report: NFT Asset Store	3
1. Introduction.....	3
2. Framework Used	3
2.1 Key Libraries and Dependencies	3
3. Project File Structure	3
4. Smart Contract ABI Integration.....	4
4.1 What is the ABI?.....	4
4.2 ABI File (NFTAssetStore.json).....	4
5. Key Functionalities	4
5.1 Wallet Connection (web3.js & MetaMask)	4
5.2 Creating an NFT Asset (CreatNFT.vue).....	5
5.3 Loading and Displaying NFT Assets (NFTAssetStore.vue)	6
5.4 Listing an NFT for Sale	6
5.5 Removing an NFT from Sale	7
5.6 Buying the NFT	7
6. Conclusion	9

Frontend Report: NFT Asset Store

1. Introduction

This report describes the frontend implementation of the NFT Asset Store. The frontend connects users to the deployed NFTAssetStore smart contract, displays NFT information, uploads NFT metadata to IPFS, and allows users to create, list, remove, and purchase NFT assets.

2. Framework Used

The frontend is built using Vue 3, which provides:

- Component-based structure for organizing the user interface.
- Reactive data binding for updating NFT and wallet information.
- Easy integration with Ethereum libraries and MetaMask.

Vite is used as the development environment for running and building the Vue application.

2.1 Key Libraries and Dependencies

- ethers.js - Connects the frontend with Ethereum and the NFTAssetStore smart contract.
- Uploads files to Pinata and retrieves NFT metadata from IPFS.
- Vue Router - Manages navigation between the NFT Asset Store and Create NFT Asset pages.
- MetaMask - Provides the user wallet and signs blockchain transactions.
- Pinata / IPFS - Stores NFT images and metadata in decentralized storage.

3. Project File Structure

```
NFT-Asset-Store/
|-- src/
|   |-- components/
|   |   |-- NFTAssetStore.vue    # NFT marketplace interface
|   |   |-- Mint.vue            # NFT creation form
|   |-- router/                  # Vue Router configuration
|   |-- NFTAssetStore.json       # Smart contract ABI
|   |-- web3.js                  # Blockchain connection logic
|   |-- App.vue                  # Root component
|   |-- main.js                  # Application entry point
|-- public/                      # Static files
|-- package.json                 # Dependencies and scripts
|-- vite.config.ts               # Vite configuration
```

4. Smart Contract ABI Integration

4.1 What is the ABI?

The ABI (Application Binary Interface) describes the functions and data structures exposed by the smart contract. The frontend uses the ABI together with the deployed contract address to call NFTAssetStore functions.

- createAsset - Creates and mints a new NFT.
- putForSale - Lists an owned NFT for sale.
- removeFromSale - Removes an owned NFT from sale.
- purchaseAsset - Purchases a listed NFT.
- loadNFTs - Returns all of the stored DigitalAsset information.

4.2 ABI File (NFTAssetStore.json)

The ABI generated from the compiled NFTAssetStore contract is included in the frontend project and is used by web3.js.

```
import contractABI from "./NFTAssetStore.json";

const provider = new ethers.BrowserProvider(window.ethereum);
const signer = await provider.getSigner();
const contract = new ethers.Contract(CONTRACT_ADDRESS, contractABI, signer);
```

5. Key Functionalities

5.1 Wallet Connection (web3.js & MetaMask)

The frontend requests access to MetaMask and creates an ethers.js provider and signer for blockchain interaction.

```
export async function connectWallet() {
  await window.ethereum.request({ method: "eth_requestAccounts" });
  const provider = new ethers.BrowserProvider(window.ethereum);
  const signer = await provider.getSigner();
  const address = await signer.getAddress();
  return { provider, signer, address };
}
```

✓ Allows users to connect MetaMask and use their Sepolia account in the application.

5.2 Creating an NFT Asset (CreatNFT.vue)

The Create NFT Asset page collects the title, traits, rarity, initial price, and image. The image is uploaded to Pinata/IPFS, followed by a JSON metadata file. The resulting metadata URI is then sent to the smart contract.

```
const imageResponse = await axios.post(
  "https://api.pinata.cloud/pinning/pinFileToIPFS",
  formData,
  { headers: pinataHeaders }
);

const imageCID = imageResponse.data.IpfsHash;
const imageURI = `ipfs://${imageCID}`;

const metadata = {
  name: this.nft.title,
  description: `NFT Asset Store digital asset - ${this.nft.title}`,
  image: imageURI,
  attributes: [
    { trait_type: "Traits", value: this.nft.traits },
    { trait_type: "Rarity", value: this.nft.rarity }
  ]
};
```

After the metadata is uploaded, the frontend converts the entered ETH price to Wei and calls `createAsset()`.

```
const metadataURI = await this.uploadToIPFS();
const { signer } = await connectWallet();
const contract = await getContract(signer);
const initialPrice = ethers.parseEther(String(this.nft.price));

const transaction = await contract.createAsset(
  this.nft.title,
  metadataURI,
  this.nft.rarity,
  this.nft.traits,
  initialPrice
);

await transaction.wait();
```

✓ Uploads the NFT image and metadata to IPFS and creates the ERC-721 asset on-chain.

5.3 Loading and Displaying NFT Assets (NFTAssetStore.vue)

The marketplace loads the total number of NFT assets from the contract and iterates through token IDs starting from 1. For each token, it retrieves the on-chain asset details, current owner, and IPFS metadata.

```
const total = await contract.totalAssets();
const totalAssets = Number(total);
const assets = [];

for (let tokenId = 1; tokenId <= totalAssets; tokenId++) {
  const asset = await contract.getAssetDetails(tokenId);
  const owner = await contract.ownerOf(tokenId);

  const metadataURL = this.getIPFSUrl(asset.metadataURI);
  const response = await axios.get(metadataURL);
  const metadata = response.data;

  assets.push({
    id: tokenId,
    title: asset.title || metadata?.name || `NFT #${tokenId}`,
    image: metadata.image ? this.getIPFSUrl(metadata.image) : "",
    sellingPrice: ethers.formatEther(asset.sellingPrice),
    forSale: asset.forSale,
    owner
  });
}
```

✓ Displays NFT title, image, rarity, traits, prices, listing status, token ID, and current owner.

5.4 Listing an NFT for Sale

Only the current NFT owner is shown the listing controls. The entered ETH selling price is converted to Wei before `putForSale()` is called.

```
const { signer } = await connectWallet();
const contract = await getContract(signer);
const priceInWei = ethers.parseEther(String(nft.listingPrice));
const transaction = await contract.putForSale(nft.id, priceInWei);

await transaction.wait();
await this.loadNFTs();
```

✓ Allows the current owner to list an NFT at a selected selling price.

5.5 Removing an NFT from Sale

When an owned NFT is already listed, the frontend provides a Remove From Sale button that calls `removeFromSale()`.

```
const { signer } = await connectWallet();
const contract = await getContract(signer);
const transaction = await contract.removeFromSale(tokenId);

await transaction.wait();
await this.loadNFTs();
```

✓ **Allows the current owner to remove an NFT from the selling list.**

5.6 Buying the NFT

A Buy button is shown when the NFT is listed and the connected wallet is not the current owner. Before sending the transaction, the frontend reads the buyer's Sepolia ETH balance and compares it with the NFT selling price.

```
const asset = await contract.getAssetDetails(nft.id);
const buyerAddress = await signer.getAddress();
const balance = await signer.provider.getBalance(buyerAddress);
const nftPrice = asset.sellingPrice;

if (balance <= nftPrice) {
  alert("Insufficient Balance!");
  return;
}

await contract.purchaseAsset.staticCall(nft.id, { value: nftPrice });

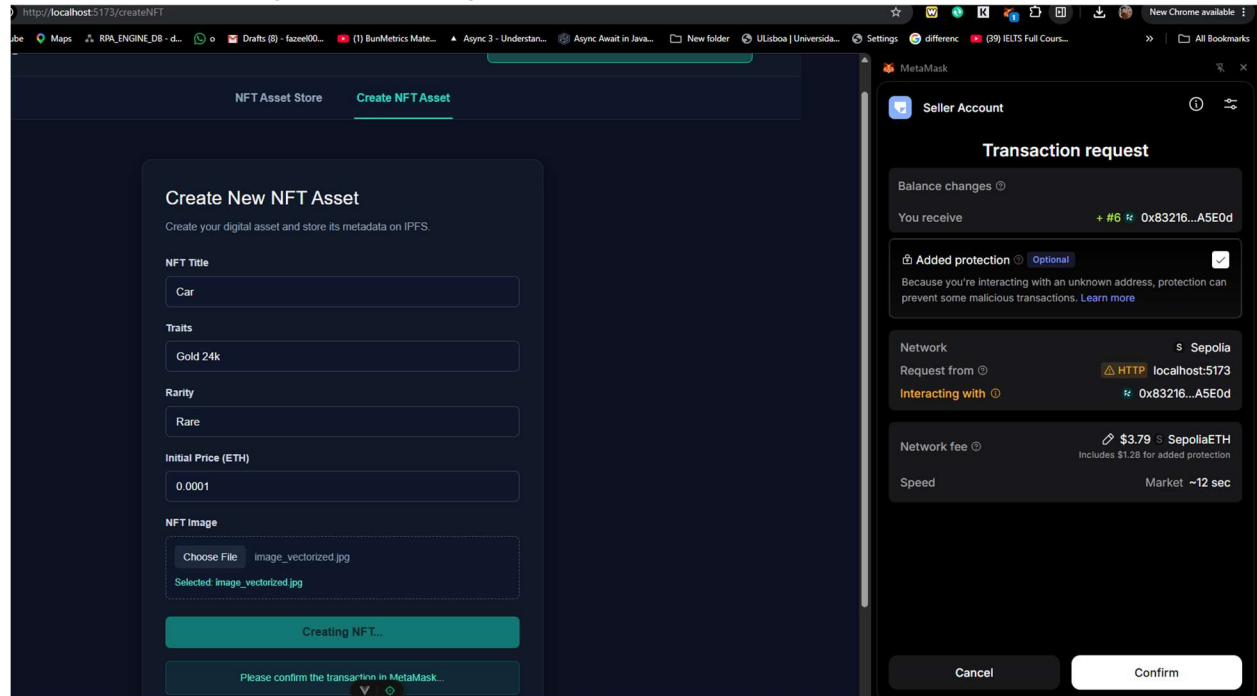
const estimatedGas = await contract.purchaseAsset.estimateGas(
  nft.id,
  { value: nftPrice }
);

const transaction = await contract.purchaseAsset(
  nft.id,
  { value: nftPrice, gasLimit: estimatedGas * 2n }
);

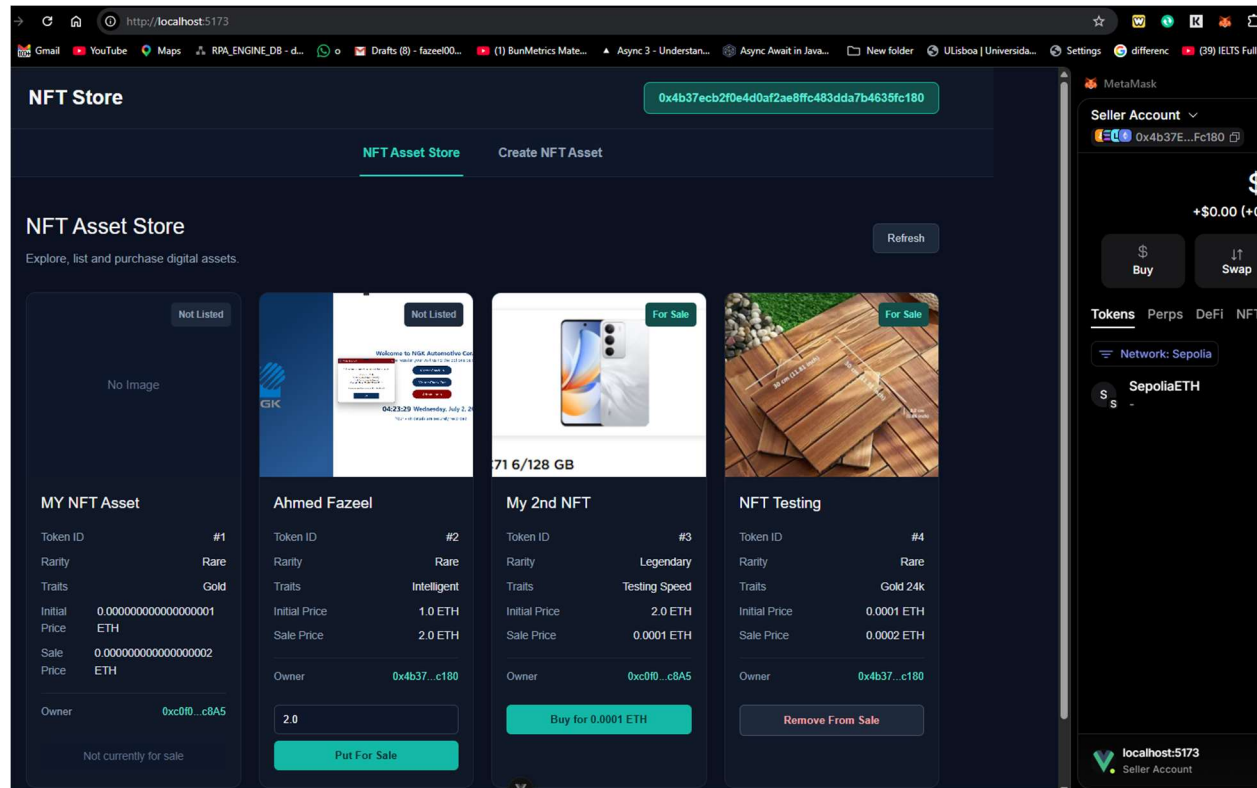
await transaction.wait();
```

✓ **Transfers NFT ownership to the buyer and sends the exact selling price to the seller through the smart contract.**

NFT Asset Creation (User Interface):



NFT Asset Store (Purchase NFT, Up For Sell, Remove From Sell):



6. Conclusion

The NFT Asset Store frontend is built with Vue 3 and connects to the NFTAssetStore ERC-721 smart contract using ethers.js and MetaMask. Pinata and IPFS are used for NFT image and metadata storage. The interface allows users to create NFT assets, view their details, list or remove owned NFTs from sale, and purchase listed NFTs using Sepolia ETH. The frontend therefore provides a complete user interface for interacting with the decentralized NFT application without using Remix or a block explorer.